Lab on Triggers and Stored Procedures

(LATE AND INCOMPLETE ASSIGNMENTS WILL NOT BE MARKED)

Instructions

1. Use **BigBookBiz.sql,** which creates the book publisher's database described in the accompanying handout. You do not need to study this database in its entirety for this lab, as only the TITLE table will be used.
2. Log on to MySQL and create the database using **BigBookBiz.sql**.

```
source /library/students/comp2521/BigBookBiz.sql
```

3. Read the introduction, do the exercises, and then read the introduction again.

Stored Procedures

A stored procedure is procedural code stored in the database. This is good for implementing business procedures, enforcing business rules, and other data constraints. So far, we have seen SQL giving us SELECT, INSERT, and UPDATE, which are non-procedural.

Advantages of stored procedures:

- Code is centrally stored and so shareable
- effective maintenance and flexibility, as you need to change only one instance of code
- Enforces data integrity
- Enforces security
- Consistent
- Data and code are stored together

Procedures are executed when **called** from an external program. It requires an explicit call for execution.

<u>Exercise 1</u>

1) Create a script called **[mymru_username].sql**;
2) Create a stored procedure to add an author to the database as follows:

```
DROP PROCEDURE add_author;
delimiter $$
CREATE PROCEDURE add_author
(
   IN id CHAR(11),
   IN last VARCHAR(40),
   IN first VARCHAR(20)
)
BEGIN
INSERT INTO author (au_id, au_lname, au_fname)
       VALUES (id, last, first);
END$$
delimiter ;
```

3) Now, you can call the procedure as follows:

```
CALL add_author('300', 'Collins', 'Suzanne');
CALL add_author('400', 'Ittyipe', 'Shoba');
```

4) Write a SELECT statement to retrieve the rows inserted.

5) Similarly, create a stored procedure to add a title to the database as follows:

```
DROP PROCEDURE add_title;

delimiter $$
CREATE PROCEDURE add_title
(
   IN tit_id CHAR(6),
   IN tit_name VARCHAR(80),
   IN publisher CHAR(4)
)
BEGIN
INSERT INTO title (title_id, title, pub_id)
        VALUES (tit_id, tit_name, publisher);
END$$
delimiter ;
```

6) Now, you can call the procedure as follows:

```
CALL add_title ("123", "About Life", "0877");
CALL add_title ("789", "Udacity", "1389");
```

7) Write a SELECT statement to retrieve the rows inserted. For simplicity's sake, only the NOT NULL fields are populated.

8) Create a **function** `find_title()` to return the title_id when the name of the title is provided.

```
DROP FUNCTION find_title;
delimiter $$
CREATE FUNCTION find_title(titleName CHAR(80))
   RETURNS char(6)
BEGIN
  DECLARE id CHAR(6);
  SELECT title_id
  INTO id
  FROM title WHERE title = titleName;
  return id;
END$$
delimiter ;
```

9) You can execute this function with the following statement.

```
SELECT find_title ("About Life") as id;
```

10) Now, we want to create a record in the `AUTHOR_TITLE` table using a stored procedure. Study the table to see what this contains. Retrieve the records from this table and order them by `title_id` and `au_ord`, and observe the data in this table.

11) Now, create a **procedure** `addAuthorTitle` to add a record to this table. Make use of the above function to find the `title_id` of the title, given the title's name.

```
DROP PROCEDURE addAuthorTitle;
delimiter $$
CREATE PROCEDURE addAuthorTitle(
IN auNbr CHAR(11),
IN titleName VARCHAR(80),
IN ordering DECIMAL(3,0),
IN royalty decimal(6,2))
BEGIN
    DECLARE aid INT;
```

```
    INSERT INTO author_title (au_id, title_id, au_ord, royaltyshare)
           VALUES (auNbr, find_title(titleName), ordering, royalty);
END$$
delimiter ;
```

12) Call the above procedure with the following statements:

```
CALL addAuthorTitle(300, "About Life", 1, 0.6);
CALL addAuthorTitle(400, "About Life", 2, 0.4);
```

13) Write a SELECT statement to retrieve the author's last name, first name, and the title's name, the author order, and the royalty share of each author for the title "`About Life`".

14) The following is an example to show OUT variables inside stored procedures:

```
DROP PROCEDURE add_author_check;
delimiter $$
CREATE PROCEDURE add_author_check
(
   IN id CHAR(11),
   IN last VARCHAR(40),
   IN first VARCHAR(20),
   IN a VARCHAR(50),
   OUT b VARCHAR(20)
)
BEGIN
IF a LIKE 'Justin Beiber%' THEN
   SET b = 'Invalid Entry!';
ELSE
   INSERT INTO author (au_id, au_lname, au_fname, address)
        VALUES (id, last, first, a);
END IF;
END$$
delimiter ;
```

15)Call the above procedure with the following statement:

```
CALL add_author_check('11', 'Gomez', 'Selena', 'Justin Beiber',
@just);
```

16) You can view the just variable of the outcome of the above code as follows:

```
select @just;
```

Triggers

A database trigger is a procedure that is stored in the database and executed on the database server by the DBMS. They are special cases of stored procedures.

The object that a database trigger is associated with is a table, and it is fired by the DBMS when an insert, update, or delete statement is executed against this table.

Coding a process as a database trigger, rather than in the client, ensures:

- It is uniformly applied – it will be performed for every application, present and future, that uses the table.

- The process only needs to be coded once; if it is changed, then there is only one copy to change. This can be a major advantage because even if only a single application performs the operation that fires the trigger, typically, it will be deployed on many user workstations, and reinstalling a new version on the client side will be a major undertaking.

- Since a database trigger is executed on the database server, when it acts on tables stored in the database, it reduces the flow of commands and data between the application and the database, compared to implementing the same task on the client side.

Triggers can contain SQL and structured SQL statements. A trigger can also invoke procedures that are stored in the database and can implicitly cause other triggers to execute.

Triggers are typically used for:

- enforcing business rules of an application;
- propagating changes through the database;
- automatically generating derived column values;
- enforcing security authorisations and constraints;
- providing transparent event logging;
- gathering statistics on table access.

Types of Triggers

Triggers are classified in **three** ways, listed below (yielding twelve types).

1. Trigger Operation

This is the SQL statement whose execution causes the DBMS to execute the trigger. It can be **UPDATE**, **INSERT,** or **DELETE**. In some implementations of triggers, execution may be limited by using a **WHEN** clause to specify a condition based on column values that a row must satisfy in order for the trigger to be fired. This is not available in the current version of MySQL.

The before and after versions of the record are accessible in the trigger by default through the references **old** and **new**. The before version of the record is the copy of the record read from the table for processing, and the after version represents the result of processing the SQL statement (this is the version of the record that will be written back to the database when the work is committed). For example, if an update statement changes the value of a column called **qoh**, then the original and updated values are available through **old.qoh** and **new.qoh**, respectively.

When a row is inserted, all old columns are null, and when a row is deleted, all new columns are null.

In summary:

SQL Statement	**old**	**new**
INSERT	Null	record being inserted
UPDATE	copy of the record before the update	copy of the record after the update
DELETE	record being deleted	Null

2. Trigger Level

This governs the number of times the trigger is executed.

- If FOR EACH ROW is specified, then the trigger is called a row trigger. A row trigger executes once for each row of the table affected by the trigger operation. For example, if an update statement with a WHERE clause updates twenty rows, then the trigger fires twenty times. **However, if the update statement affects no rows (because its where condition is satisfied by none), then the trigger is not fired.**

- If FOR EACH ROW is not specified, then the trigger is called a statement trigger and is executed only once for the trigger operation. If an update statement updates twenty rows, the trigger is fired only once. **However, if the update statement affects no rows, then the trigger is still fired once. Currently, MySQL only implements FOR EACH ROW triggers.**

3. **Trigger Time**

   This specifies when the trigger is executed in relation to the database event that causes it to fire. A trigger can be invoked either before or after the trigger operation is executed. The after record can be changed only in an after trigger.

Naming Triggers

A trigger **<u>naming convention</u>** that includes a three-letter suffix or prefix code indicating its type is often used. The code is formed as follows:

- The first letter of the time when the trigger fires – B for Before or A for After.
- The first letter of the operation that fires the trigger – D for Delete, I for Insert or U for Update.
- The first letter of the level at which the trigger fires – R for Row level or S for Statement level.

For example, BIR would be used for a row-level trigger that fires before an insert operation.

<u>Exercise 2</u>

1) Add a CREATE TABLE statement to create a table `book_price_audit`, with columns
   a. title_id char(6)
   b. type char(12)
   c. old_price numeric(6,2)
   d. new_price numeric(6,2)

2) <u>Do not</u> define a primary key. A primary key is usually not defined for a table that is simply used to log or transfer data.

3) The trigger definition statement is given by the following syntax:

```
CREATE TRIGGER <trigger name>
                        trigger name
BEFORE | AFTER
        INSERT | DELETE | UPDATE
        ON <table name>                          trigger event
        FOR EACH ROW

        BEGIN           trigger action
        DECLARE …
        ...
        END;
```

4) Now, you are about to create a trigger on the **TITLE** table to insert a row in the **BOOK_PRICE_AUDIT** table whenever a price is increased by 10% or more.

5) Name the trigger **audit_book_price_BUR**. From the naming convention, try and understand the operation, level and timing of the trigger. Thus, it is to be defined as a **b**efore **u**pdate **r**ow trigger.

6) Code the trigger on **TITLE** as follows:

```
delimiter $$
CREATE TRIGGER audit_book_price_BUR
BEFORE UPDATE
ON title
FOR EACH ROW
BEGIN
   IF (new.price / old.price >= 1.1) THEN
         INSERT INTO book_price_audit
   VALUES(new.title_id, new.type, old.price, new.price);
END IF;
END$$
delimiter ;
```

7) It is always good to drop a trigger before you create one using the statement below:

```
drop trigger audit_book_price_BUR;
```

8) The insert statement in the trigger

```
insert into book_price_audit
values (new.title_id, new.type, old.price, new.price);
```

is coded within the body of the trigger.

A trigger operates internally on tables in the database, and no COMMIT is required.

9) If no error is returned from the database server, then the trigger has been successfully compiled and stored in the database.

10) This trigger will be executed whenever a user increases a book price by more than 10%. The DBMS will fire the trigger.

11) Change the price of the title of PC8888 from $20.00 to $21.00 (less than 10%) and the price of the title BU1032 from $19.99 to $25.00 (10% or more).

12) Using SQL, display the data in the **BOOK_PRICE_AUDIT** table to verify that **one** audit record has been written.

<u>Exercise 3</u>

Suppose you wanted to have a surrogate key in the `book_price_audit` table (this is just an exercise; you would likely leave it without a key). Using an ALTER TABLE command, add an audit_nbr column to the table definition. The audit_nbr is a sequential number, starting at 1. (Of course, you could do this with an AUTOINCREMENT field, but for this case, do it with a trigger. There are times when you need to do this anyway because you cannot have two auto-increment fields in the same table.

Code the trigger on **BOOK_PRICE_AUDIT** as follows:

```
create trigger generate_audit_nbr_BIR
before insert
on BOOK_PRICE_AUDIT
for each row
BEGIN
   /* Find the current sequence number */
   /* Set the new number.
END;
```

1) Since the audit_book_price_BUR trigger <u>no longer inserts a value for every column</u>, the list of columns

```
(title_id, type, old_price, new_price)
```

has to be added to the insert statement in the trigger.

Therefore, the insert statement on the TITLE table written in Exercise 1 now has to be changed to

```
insert into book_price_audit (title_id, type, old_price, new_price)
     values (new.title_id, new.type, old.price, new.price);
```

Drop the old trigger on TITLE and create a new one applying the change.

In general, the INSERT trigger of a table is the appropriate place to generate column values for new records because the code will be executed no matter of where the INSERT statement comes. Notice that, in this case, the INSERT statement is executed internally in the database from another database trigger.

A surrogate key can also be generated by a database trigger when a form is inserting a record, and the user has no need to know what the surrogate key value is.

2) Now using SQL, change three prices of titles, two of them increase by 10% or more and one by less than 10%. Then use SQL to examine the BOOK_PRICE_AUDIT table. Observe the generated sequence in the audit_nbr column.

<u>Exercise 4</u>

1) Looking back into BigBookBiz.sql, you will find at the end of the script file an ALTER TABLE statement to add a total_income column to the TITLE table (this is a derived value – it is just being done to get another trigger example). As you executed the script file, this statement was already executed, and therefore, the column total_income already exists in the table.

2) The ALTER TABLE statement is followed by an UPDATE command in order to compute the total_income field for all existing rows in the table. As you have changed prices for several records using UPDATE commands in Exercises 1 and 2, it is wise to re-run the statement once again to ensure that the total_income is now up-to-date.

```
UPDATE title
SET total_income = price * ytd_sales;
```

3) Examine the table to ensure that values for the total_income field have been reset to newly computed values, adhering to the updates that were made earlier to the price column.

Code a SELECT statement to display the price, ytd_sales, and total_income for the records with title_id equal to BU1111 or MC2222.

4) Ideally, the total_income should be computed when either the price or the ytd_sales change in order to maintain consistent data. To do this, drop the old trigger on TITLE and create a new one applying the change. Include code (you can modify the earlier code) to update the total_income column when either the price or the ytd_sales columns change.

5) For the book MC2222, change the year-to-date sales from 2032 to 4000, and for the book BU1111, change the price from $11.95 to $21.99.

6) Code a SELECT statement to display the price, ytd_sales, and total_income for the records with title_id equal to BU1111 or MC2222 to view the change.

Enabling and Disabling a Trigger

A trigger is enabled by default when it is created. It can be disabled so that it is retained in the database but does not fire when the trigger operation is executed with the SQL ALTER TRIGGER statement.

For example, to disable a trigger, the command is:

alter trigger <trigger_name> **disable**;

ALTER TRIGGER can also be used to enable a trigger.

Submitting Your Work

Submit your `.sql` file to the Blackboard drop box provided.